Number Theory: Primes and Factors

(Based on the CSES Competitive Programming Handbook)

1. The Fundamental Theorem of Arithmetic

Every integer $n > 1$ can be uniquely represented as a product of prime numbers. This is the foundation of all factor-related algorithms.

Notation: $n = p_1^{a_1} \cdot p_2^{a_2} \cdot p_3^{a_3} \dots$

- Where $p_1, p_2, p_3 \dots$ represent the unique **prime factors** (e.g., 2, 3, 5).
- Where $a_1, a_2, a_3 \dots$ represent the **powers or counts** of those specific primes.

Example: For $n = 84$: $84 = 2^2 \cdot 3^1 \cdot 7^1$ Here, the primes are $p_1 = 2, p_2 = 3, p_3 = 7$ and their powers are $a_1 = 2, a_2 = 1, a_3 = 1$.

2. Number of Factors (Divisors)

Let $\tau(n)$ or $d(n)$ be the total number of positive factors a number n has.

The Formula: $d(n) = (a_1 + 1) \cdot (a_2 + 1) \cdot (a_3 + 1) \dots$

*(Reminder: $a_1, a_2, a_3 \dots$ are the **exponents/powers** of the prime factors of n)*

The Reasoning (Combinatorics): To build a factor of n, you must choose how many of each prime to include. Look at $n = 2^2 \cdot 3^1$. Here $a_1 = 2$ and $a_2 = 1$.

- How many 2s can you use to build a factor? You can use zero 2s, one 2, or two 2s. That is $(a_1 + 1) = (2 + 1) = 3$ choices.
- How many 3s can you use? You can use zero 3s or one 3. That is $(a_2 + 1) = (1 + 1) = 2$ choices.
- Since the choices are independent, you multiply them: $3 \times 2 = 6$ total factors.

Example ($n = 84$): $84 = 2^2 \cdot 3^1 \cdot 7^1$ (Powers are $a_1 = 2, a_2 = 1, a_3 = 1$)
$d(84) = (2 + 1) \cdot (1 + 1) \cdot (1 + 1) = 3 \cdot 2 \cdot 2 = 12$ factors.

C++ Code (Using pre-computed primes / SPF):

```
long long num_of_factors(long long n) {
    long long factors = 1;
    // Assuming you have an spf[] (Smallest Prime Factor) array precomputed
    while (n > 1) {
        int p = spf[n];
        int count = 0; // This 'count' represents our powers (a1, a2, etc.)
```

```
        while (n % p == 0) {
            count++;
            n /= p;
        }
        factors *= (count + 1); // Multiplying by (a_i + 1)
    }
    return factors;
}
```

3. Sum of Factors

Let $\sigma(n)$ be the sum of all positive factors of n.

The Formula: $\sigma(n) = \left(\frac{p_1^{a_1+1}-1}{p_1-1}\right) \cdot \left(\frac{p_2^{a_2+1}-1}{p_2-1}\right) \cdot \ldots$

(Reminder: p is the prime base, a is its exponent)

The Reasoning (Algebra): If you want to add up every single factor combination of n, it perfectly factors into a product of sums: Sum = $(1 + p_1 + p_1^2 + \cdots + p_1^{a_1}) \cdot (1 + p_2 + p_2^2 + \cdots + p_2^{a_2}) \ldots$ If you multiply those brackets out, you will generate exactly every factor exactly once! Notice that the inside of each bracket is a **Geometric Progression (GP)**. The formula for the sum of a GP is $\frac{r^{k+1}-1}{r-1}$. Substituting our primes and powers gives the final formula.

Example ($n = 12$): $12 = 2^2 \cdot 3^1$ (Primes $p_1 = 2, p_2 = 3$. Powers $a_1 = 2, a_2 = 1$) Using the formula method: $\left(\frac{2^{2+1}-1}{2-1}\right) \cdot \left(\frac{3^{1+1}-1}{3-1}\right) = \left(\frac{8-1}{1}\right) \cdot \left(\frac{9-1}{2}\right) = 7 \cdot 4 = 28$. *(Manual check: factors of 12 are 1, 2, 3, 4, 6, 12. Sum = 28).*

C++ Code (Using pre-computed primes / SPF):

```
long long sum_of_factors(long long n) {
    long long sum = 1;
    while (n > 1) {
        int p = spf[n];
        long long current_power = p;
        while (n % p == 0) {
            current_power *= p; // This will calculate p^(a+1)
            n /= p;
        }
        // Formula: (p^(a+1) - 1) / (p - 1)
        sum *= (current_power - 1) / (p - 1);
    }
    return sum;
}
```

4. Product of Factors

Let $\mu(n)$ be the product of all positive factors of n.

The Formula: $\mu(n) = n^{d(n)/2}$ *(Where $d(n)$ is the total number of factors we found in Section 2)*

The Reasoning (Symmetry): Write out all the factors of a number in increasing order. You will notice that if you pair the smallest factor with the largest, they multiply to n. The second smallest and second largest also multiply to n. For $n = 12$, factors are: `1, 2, 3, 4, 6, 12`. Pairs: $(1 \times 12) = 12$, $(2 \times 6) = 12$, $(3 \times 4) = 12$. Since there are $d(n)$ total factors, there are exactly $\frac{d(n)}{2}$ pairs. Each pair multiplies to n. Therefore, the product is n multiplied by itself $\frac{d(n)}{2}$ times.

Note for perfect squares (like $n = 16$): They have an odd number of factors. The middle factor is exactly $\sqrt{n}$. The formula still works perfectly because $n^{\text{odd}/2} = (\sqrt{n})^{\text{odd}}$.

C++ Code (Handling modulo arithmetic - frequently tested in CSES): *Note: If calculating modulo $MOD = 10^9 + 7$, you must compute $d(n)$ modulo $(MOD - 1)$ due to Fermat's Little Theorem.*

```
long long product_of_factors(long long n, long long d_n, long long MOD) {
    // If d(n) is odd, n is a perfect square. Be careful with dividing by 2!
    // Often it's safer to use modular exponentiation:
    // This assumes you have a fast exponentiation function: power(base, exp, moc

    // Simplest conceptual approach if avoiding modulo complexities:
    // (Ensure you handle perfect squares correctly if d_n is odd)
    if (d_n % 2 == 1) {
        long long root = sqrt(n);
        return power(root, d_n, MOD);
    } else {
        return power(n, d_n / 2, MOD);
    }
}
```

5. Algorithmic Techniques (For your TRD)

A. Trial Division (The $\sqrt{n}$ method)

Concept: You only need to check divisors up to $\sqrt{n}$. **Reasoning:** If $n = a \cdot b$, one of the factors must be $\leq \sqrt{n}$ and the other must be $\geq \sqrt{n}$. If you don't find a factor by $\sqrt{n}$, n is prime. **When to use:** N is massive (10^{12}), but Queries are very few ($Q = 1$ to 100). **Complexity:** $O(\sqrt{n})$ per query.

```
long long count_factors_sqrt(long long n) {
    long long count = 0;
    for (long long i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            count++;            // i is a factor
            if (i * i != n) {
                count++;        // (n/i) is also a factor
            }
        }
    }
    return count;
}
```

B. SPF (Smallest Prime Factor) Array - *Crucial for CSES*

Concept: A modification of the Sieve of Eratosthenes. Instead of storing `true/false`, let `spf[i]` store the smallest prime number that divides `i`. **Why it's powerful:** If you want to factorize a large number X, you don't need a `for` loop up to $\sqrt{X}$. You just jump from prime factor to prime factor in $O(\log X)$ time. **When to use:** N is up to 10^7 (fits in memory) and Queries are huge (10^5). **Complexity:** Precomputation $O(N \log \log N)$, Answering each query $O(\log X)$.

```
const int MAXN = 1000005; // Change based on problem constraints
int spf[MAXN];

void sieve() {
    for (int i = 2; i < MAXN; i++) {
        spf[i] = i; // Initialize: smallest prime factor of a number is itself
    }
    for (int i = 2; i * i < MAXN; i++) {
        if (spf[i] == i) { // If i is prime
            for (int j = i * i; j < MAXN; j += i) {
                if (spf[j] == j) {
                    spf[j] = i; // Mark the smallest prime factor
                }
            }
        }
    }
}
```

Note: Always call `sieve()` exactly ONCE in your `main()` before processing any queries!